Softwarearchitektur

Software Engineering II

S.A.F.T. Corp.

Secret Audio File Transformer

DHBW

DHBW Mannheim
29. Januar 2026

Änderungshistorie

Revision	Datum	Autoren	Beschreibung
1.0	05.03.2026	T,J	A: 1-3

Abkürzungen: Hinzugefügt/Added (A), Änderung/Changed (C), Löschung/Deleted (D),
Gerrit (G), Raphael (R), Tu (T), Paul (P), Johannes (J)

Inhaltsverzeichnis

1 Einleitung----------1
2 Hauptteil---------- 2
2.1 Struktur---------- 2
2.2 Interaktion----------3
2.3 Technologieauswahl----------3
2.4 Begründung der Architektur---------- 3
3. Zusammenfassung---------- 4

1 Einleitung

In diesem Dokument wird die Softwarearchitektur der SAFT-Anwendung im Detail beschrieben. Zuerst wird der strukturelle Aufbau der komponentenbasierten Architektur erläutert. Dann werden Kommunikationswege und Schnittstellen, durch die die Komponenten verbunden sind, beschrieben. Abschließend wird die Auswahl der genutzten Technologien (z.B. Vue.js) und die Wahl der daraus abgeleiteten Architektur begründet.

2 Hauptteil

Die Architektur der SAFT-Anwendung ist als **komponentenbasierte Web-Applikation** konzipiert. Sie folgt dem Prinzip der **funktionalen Dekomposition**, um eine klare Trennung der Zuständigkeiten (Separation of Concerns) zu erreichen und die Wartbarkeit sowie Erweiterbarkeit des Systems sicherzustellen

2.1 Struktur

Das System ist in drei logische Bereiche unterteilt, wie in der folgenden Übersicht der komponentenbasierten Struktur dargestellt:

1. UI Components: Diese Ebene beinhaltet wiederverwendbare Oberflächenelemente wie die *Navbar*, *Cards* zur Inhaltsdarstellung und das *Canvas*-Element für die visuelle Audio-Repräsentation.
2. Views: Über ein zentrales Routing werden die verschiedenen Anwendungszustände (*UploadView*, *DownloadView*, *CanvasView*) gesteuert. Jede View bündelt dabei eine spezifische Teilmenge der Gesamtfunktionalität.
3. Services: In dieser Schicht ist die Business-Logik gekapselt. Hierzu gehören spezialisierte Dienste für die Datenverarbeitung wie FFT (Fast Fourier Transform), Reverse FFT sowie Module für den Import und Export (siehe Abb. 1)

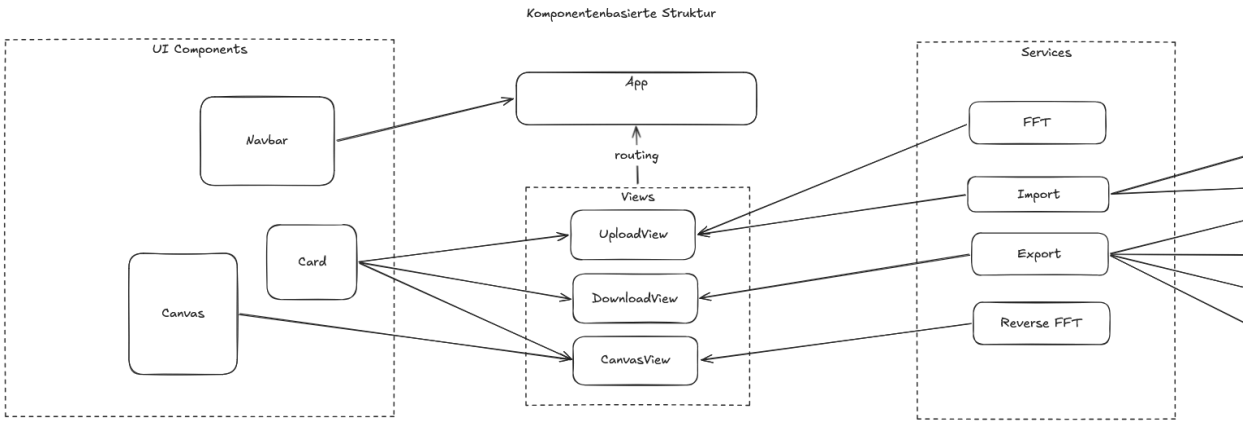

Abb. 1: Visuelle Darstellung der Komponenten und ihre Relation zueinander
(Excalidraw Source)

2.2 Interaktion

Die Komponenten interagieren über definierte Schnittstellen und Kommunikationswege miteinander:

- Datenfluss: UI-Komponenten wie das *Canvas* sind an die jeweiligen *Views* gekoppelt. Die *Views* wiederum nutzen die Dienste der *Service-Schicht*.
- System-Integration: Die Services agieren als Vermittler zum Betriebssystem des Clients (*Client OS*). So greift der *Import-Service* auf das Dateisystem oder das Mikrofon zu, während der *Export-Service* Schnittstellen zu Plattformen wie WhatsApp, Telegram oder den lokalen Lautsprechern bedient.
- Zustandsmanagement: Die reaktive Datenbindung des Frameworks stellt sicher, dass Statusänderungen in den Services (z. B. Abschluss einer Transformation) unmittelbar in der UI reflektiert werden.

2.3 Technologieauswahl

Für die Umsetzung wurden moderne Webtechnologien gewählt, um eine hohe Interoperabilität zu gewährleisten:

- Frontend-Framework: Vue.js dient als Basis für die komponentenbasierte Architektur.
- Deployment-Modell: Die Anwendung wird als Progressive Web App (PWA) realisiert, wodurch sie ohne Anpassungen auf verschiedenen Endgeräten lauffähig ist.
- Infrastruktur: Das Hosting und die automatisierte Bereitstellung erfolgen über GitHub Actions und GitHub Pages.

2.4 Begründung der Architektur

Die Entscheidung für diese spezifische Architektur basiert maßgeblich auf den zentralen Qualitätsanforderungen an das System sowie der gewählten technologischen Basis. Durch die komponentenbasierte Struktur in Verbindung mit dem Vue.js-Framework wird eine hohe Wartbarkeit und Erweiterbarkeit erreicht, da funktionale Einheiten gekapselt und so unabhängig voneinander getestet oder modifiziert werden können. Dies folgt dem Architektur-Grundsatz der Separation of Concerns, um die Komplexität durch eine klare Trennung der Zuständigkeiten beherrschbar zu machen. Vue.js bietet hierfür eine optimale Grundlage, da es neben einer reaktiven Datenbindung auch eine vergleichsweise flache Lernkurve aufweist und somit eine effiziente Entwicklung dynamischer Weboberflächen ermöglicht.

Ein wesentlicher Faktor der Architekturplanung ist zudem die Interoperabilität, welche durch die Umsetzung als Progressive Web App (PWA) sichergestellt wird. Dieser Ansatz ermöglicht es, die Anwendung plattformunabhängig auf verschiedensten Endgeräten zu betreiben, ohne dass systemnahe Anpassungen für das jeweilige Client-Betriebssystem erforderlich sind. Die funktionale Dekomposition in spezialisierte Schichten - von den UI-Komponenten über die Views bis hin zu den Services für die Audio-Transformation wie FFT - stellt dabei sicher, dass nicht-funktionale Anforderungen wie Performance, Verlässlichkeit und Sicherheit bereits im strukturellen Design des Systems verankert sind. Abgerundet wird das Architekturkonzept durch den Einsatz moderner CI/CD-Prozesse via GitHub Actions und ein effizientes Hosting über GitHub Pages, was einen stabilen und automatisierten Bereitstellungszyklus gewährleistet.

3. Zusammenfassung

Die Softwarearchitektur von SAFT folgt einem komponentenbasierten Aufbau. Sie ist dabei grob in drei logische Bereiche: UI Components, Views und Services aufgeteilt, welche sowohl die Entwicklung der grafischen Oberfläche, als auch die Verknüpfung und Funktionsweise der zugrundeliegenden Logik bestimmen. Die Auswahl der Frontend-Frameworks Vue.js ist ein Resultat aus unseren technischen Anforderungen und persönlichen Erfahrungen. Die Architektur-Entscheidung ist ein direktes Resultat aus der Wahl des Frameworks.